--Slip 1

Q.1) Take multiple files as Command Line Arguments and print their inode numbers and file types [10 Marks]

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
#include <int types.h>
#include <string.h>
#include <errno.h>

const_file_type(mode_t mode)
{
       if (S_ISBLK(mode)):  return "Block device";
       if (S_ISCHR(mode)):  return "Character device";
       if (S_ISDIR(mode):  return "Directory";
       if (S_ISIFO(mode)):  return "FIFO/pipe";
       if (S_ISLNK(mode)):  return "Symbolic link";
       if (S_ISREG(mode)):  return "Regular file";
       if (S_ISSOCK(mode)): return "Socket\n");
               return"Unknown\n";
    }
int main(int argc, char *argv[])
{
         struct stat st;
int 1;
        if (argc < 2)
{
        fprintf(stderr, "Usage: %s <file1> [file2] ...\n", argv[0]);
        return;
    }
    for (int i = 1; i < argc; i++) {
if (stat [argc i],&st)==-1)
{
Fprint(stderr,"%s: cannot stat %s:%s\n",arg[0],arg[i],strerror(error));
Continue;
}
Printf("%s:inode =%\u,type=
%s\n",argv[i],(uintmax_t)st.st_ino,file_type(st.st_mode));
}
Return 0;
}
```

Q.2) Write a C program to send SIGALRM signal by child process to parent process and parent process make a provision to catch the signal and display alarm is fired.(Use Kill, fork, signal and sleep system call)

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h> // Include for wait()

void sigalrm_signal(int sig) {
    printf("Parent: Alarm has fired!\n");
}

int main() {
    pid_t pid;
    signal(SIGALRM, sigalrm_signal);

    pid = fork();
    if (pid < 0) {
        printf("fork failed\n");
        exit(1);
    }

    if (pid == 0) {
        printf("Child: Waiting for 3 second before sending SIGALARM to 
parent.\n");
        sleep(3);
        kill(getppid(), SIGALRM);
        exit(0);
    } else {
        printf("Parent: Waiting for child to send signal...\n");
        wait(NULL);
    }
    return 0;
}
```